You are absolutely right. The environment is just concrete and fog until the people and the traffic fill it. In a game deeply rooted in regional satire, if the player drives from the rusted mills of Cascade into the manicured hedges of Cape Cobb and the pedestrians and cars don't *immediately* and drastically change, the illusion dies.
We need to inject absolute urgency into this. The map must be instantly identifiable by simply looking at the traffic and the NPCs walking down the sidewalk.
Here is the blueprint for the **Geo-Aware Traffic & Pedestrian Spawner**.

## 1. The Regional Identification Strategy

To make the South Coast feel accurate, the AI spawner cannot use a global randomizer. It must use a highly aggressive **Zone-Weighting System** tied to the specific districts.

- **Cascade (Fall River):** 80% spawn rate for rusted "Townie" sedans and "Linguiça" mopeds. Pedestrians are heavily weighted toward older Azorean men in tracksuits or heavy sweaters, women carrying grocery bags, and Azorean Syndicate members near the bakeries.
- **New Sefton (New Bedford):** 70% spawn rate for commercial vehicles (fish trucks, union vans). Near the South End, the spawner aggressively populates Cape Verdean "Crioulo Crews" and customized "Açoreana" lowriders.
- **Cape Cobb (Cape Cod):** 90% spawn rate for "Codfish 40" yachts near the water, preppy pedestrians in boat shoes and Nantucket reds, and "Bourne" commuter cars.
- **The Gridlock Feature:** The Burns and Sachem bridges are permanently jammed. The traffic spline here does not flow; it is an active, stationary parking lot of AI vehicles that the player must weave through or use a moped to bypass.

## 2. The Architectural Blueprint (The Spawner)

This script uses a Scriptable Object architecture. You will create a RegionProfile for each of the four districts. When the player crosses a boundary line (e.g., driving over the Verde Bridge), the SpawnManager swaps the profile and instantly changes what it pulls from the object pool.

```
// File: /Systems/AI/RegionProfile.cs
using UnityEngine;
using System.Collections.Generic;

[CreateAssetMenu(fileName = "NewRegionProfile", menuName = "South Coast/Region Profile")]
public class RegionProfile : ScriptableObject
{
    public string regionName; // e.g., "Cascade", "Cape Cobb"

    [Header("Traffic Spawns (Weighted)")]
    public List<SpawnWeight<GameObject>> vehiclePrefabs;

    [Header("Pedestrian Spawns (Weighted)")]
    public List<SpawnWeight<GameObject>> pedestrianPrefabs;

    [Header("Region Specifics")]
    public bool forceBridgeGridlock; // If true, vehicles spawn stationary on bridge splines
```

```
    public float mopedSpawnMultiplier; // Cranks up "Linguiça" spawns in Cascade
}

[System.Serializable]
public struct SpawnWeight<T>
{
    public T prefab;
    [Range(0, 100)] public int spawnProbability;
}

// File: /Systems/AI/DynamicSpawner.cs
using UnityEngine;

public class DynamicSpawner : MonoBehaviour
{
    public static DynamicSpawner Instance { get; private set; }

    [SerializeField] private RegionProfile currentRegion;
    [SerializeField] private Transform playerTransform;
    [SerializeField] private float spawnRadius = 150f; // Spawn out of player sight

    private void Awake()
    {
        if (Instance == null) Instance = this;
    }

    // Called by a trigger volume when the player enters a new district
    public void UpdateRegion(RegionProfile newRegion)
    {
        if (currentRegion == newRegion) return;
        currentRegion = newRegion;
        Debug.Log($"[Spawner] Crossed into {currentRegion.regionName}. Updating AI
demographics.");

        // Optional: Gradually despawn out-of-sight old models to flush the memory pool
    }

    private void Update()
    {
        // Spawning logic tick (Simplified for blueprint)
        // 1. Check object pool for available AI/Vehicles
        // 2. Find a valid NavMesh point or Traffic Spline out of camera frustum
        // 3. Roll a random number against currentRegion.vehiclePrefabs weights
        // 4. Instantiate the locally accurate asset
    }

    public GameObject GetWeightedPrefab(List<SpawnWeight<GameObject>> spawnList)
    {
```

```
        int totalWeight = 0;
        foreach (var item in spawnList) totalWeight += item.spawnProbability;

        int randomRoll = Random.Range(0, totalWeight);
        foreach (var item in spawnList)
        {
            if (randomRoll < item.spawnProbability) return item.prefab;
            randomRoll -= item.spawnProbability;
        }
        return spawnList[0].prefab; // Fallback
    }
}
```

## 3. The Pedestrian "Flavor" State Machine

To make the map urgent and identifiable, the NPCs cannot just walk aimlessly. We need to attach an AIBrain script to the pedestrians that forces localized behaviors based on the WeatherController we built earlier.

- **State: Default.** Walking, holding a cup of "Dunkies" coffee, or arguing loudly in a non-rhotic accent ("pahk the cah").
- **State: Weather Panic.** If the WeatherController broadcasts the Noreaster state, pedestrian AI immediately switches. Their walk speed increases to a jog, umbrellas deploy, and audio barks trigger referencing the legendary "Bread and milk before the storm. Always. This is not a drill.".
- **State: Local Hostility.** If you bump into a Cape Cod local, they sneer. If you bump into a union dockworker in New Sefton, they swing on you.

This framework guarantees that the second you cross the city line, the entire demographic makeup of the streets shifts to accurately reflect the real-world South Coast class divide.

This is where the South Coast map stops being a static 3D model and becomes a breathing, reactive, and highly cynical simulation.

To achieve this, every pedestrian must operate as an independent state machine. They need to know *where* they are, *what* the weather is doing, and *who* just bumped into them so they can react appropriately—
whether that’s by pulling an umbrella and jogging, swinging a fist, or making a smug comment with a performative "hard R".

# 4. Pedestrian State Machine**.

### 1. The Regional & State Definitions

First, we define the states the pedestrian can be in and the regional identity they inherit from the DynamicSpawner when instantiated.

```csharp
using UnityEngine;

public enum PedestrianState
{
    Wander,
    Idle,
    WeatherPanic,
    Hostile,
    Fleeing
}

public enum RegionIdentity
{
    Cascade,       // "The Spindle City" - Working class, Portuguese influence[span_1](start_span)[span_1](end_span)[span_2](start_span)[span_2](end_span)
    NewSefton,     // "The Whaling City" - Union muscle, dockworkers[span_3](start_span)[span_3](end_span)[span_4](start_span)[span_4](end_span)
    CapeCobb,      // "The Forearm" - Old money, boat shoes, yacht clubs[span_5](start_span)[span_5](end_span)[span_6](start_span)[span_6](end_span)
    Brockmore      // "Sole City" - Boxer country, tough crowds[span_7](start_span)[span_7](end_span)[span_8](start_span)[span_8](end_span)
}

```

### 2. The Core State Machine (PedestrianAI.cs)

This script sits on the NPC prefab alongside a NavMeshAgent and an Animator. It listens to the global weather broadcasts and handles interactions with the player.

```csharp
using UnityEngine;
using UnityEngine.AI;
using System.Collections;

[RequireComponent(typeof(NavMeshAgent), typeof(Animator), typeof(AudioSource))]
public class PedestrianAI : MonoBehaviour
{
    [Header("Identity")]
    public RegionIdentity myRegion;
    private PedestrianState currentState;

    [Header("Components")]
    private NavMeshAgent agent;
    private Animator animator;
    private AudioSource audioBarker;

    [Header("Navigation")]
```

```
[SerializeField] private float wanderSpeed = 1.2f;
[SerializeField] private float panicSpeed = 3.5f;
[SerializeField] private float wanderRadius = 20f;

private void Awake()
{
    agent = GetComponent<NavMeshAgent>();
    animator = GetComponent<Animator>();
    audioBarker = GetComponent<AudioSource>();
}

private void OnEnable()
{
    // Subscribe to the global weather broadcast
    WeatherController.OnWeatherChanged += HandleWeatherShift;
    SetState(PedestrianState.Wander);
}

private void OnDisable()
{
    WeatherController.OnWeatherChanged -= HandleWeatherShift;
}

private void Update()
{
    // Basic State Router
    switch (currentState)
    {
        case PedestrianState.Wander:
            HandleWandering();
            break;
        case PedestrianState.WeatherPanic:
            HandlePanic();
            break;
        case PedestrianState.Hostile:
            // Chase or attack player
            break;
    }

    // Sync Animator
    animator.SetFloat("Speed", agent.velocity.magnitude);
}

private void SetState(PedestrianState newState)
{
    currentState = newState;

    if (newState == PedestrianState.Wander)
```

```
        {
            agent.speed = wanderSpeed;
            animator.SetBool("IsPanicking", false);
        }
        else if (newState == PedestrianState.WeatherPanic)
        {
            agent.speed = panicSpeed;
            animator.SetBool("IsPanicking", true);
            // Trigger the umbrella animation layer if they have one
            animator.SetTrigger("DeployUmbrella");
        }
    }

    // --- WEATHER LOGIC --- //

    private void HandleWeatherShift(WeatherModifiers weather)
    {
        // If a massive Nor'easter or heavy coastal rain rolls in
        if (weather.isStorming)
        {
            SetState(PedestrianState.WeatherPanic);
            TriggerWeatherBark();
        }
        else if (currentState == PedestrianState.WeatherPanic && !weather.isStorming)
        {
            // Storm passed, go back to normal
            SetState(PedestrianState.Wander);
        }
    }

    private void TriggerWeatherBark()
    {
        // 1-in-4 chance they yell the legendary local phrase when a storm hits
        if (Random.Range(0, 4) == 0)
        {
            Debug.Log($"[Audio Bark]: 'Bread and milk before the storm. Always. This is not a
drill.'[span_9](start_span)[span_10](start_span)"[span_9](end_span)[span_10](end_span));
            // audioBarker.PlayOneShot(breadAndMilkClip);
        }
    }

    // --- NAVIGATION LOGIC --- //

    private void HandleWandering()
    {
        if (!agent.hasPath || agent.remainingDistance < 0.5f)
        {
            Vector3 randomDirection = Random.insideUnitSphere * wanderRadius;
```

```
            randomDirection += transform.position;

            NavMeshHit hit;
            if (NavMesh.SamplePosition(randomDirection, out hit, wanderRadius, 1))
            {
                agent.SetDestination(hit.position);
            }
        }
    }

    private void HandlePanic()
    {
        // Find the nearest cover or interior building node and sprint to it
        // If none found, just run faster between waypoints
        HandleWandering();
    }

    // --- REGIONAL INTERACTION (THE "BUMP" REACTION) --- //

    private void OnCollisionEnter(Collision collision)
    {
        if (collision.gameObject.CompareTag("Player") && currentState != PedestrianState.Hostile)
        {
            ReactToPlayerBump();
        }
    }

    private void ReactToPlayerBump()
    {
        // Stop walking immediately
        agent.isStopped = true;
        transform.LookAt(GameObject.FindGameObjectWithTag("Player").transform);

        // Branch reaction entirely based on the region they spawned in
        switch (myRegion)
        {
            case RegionIdentity.Cascade:
                // Working class, deeply loyal, short-tempered. High chance of hostility.
                if (Random.value > 0.5f)
                {
                    SetState(PedestrianState.Hostile);
                    Debug.Log("[Audio Bark]: Non-rhotic yell: 'Watch the jacket, guy! I'll dump ya in the
Quequechan!'");
                    animator.SetTrigger("SwingFist");
                }
                break;

            case RegionIdentity.NewSefton:
```

```
            // Scallopers and union
dockworkers[span_11](start_span)[span_11](end_span)[span_12](start_span)[span_12](end_sp
an). They don't back down.
            SetState(PedestrianState.Hostile);
            Debug.Log("[Audio Bark]: 'You want a problem down the piers, kid?'");
            animator.SetTrigger("Shove");
            break;

        case RegionIdentity.CapeCobb:
            // Old money, performative
snobs[span_13](start_span)[span_13](end_span)[span_14](start_span)[span_14](end_span).
They won't fight, they just insult you.
            SetState(PedestrianState.Fleeing);
            Debug.Log("[Audio Bark]: Hard 'R' pronunciation: 'Keep your hands off my resources,
you mainland riffraff!'[span_15](start_span)"[span_15](end_span));
            animator.SetTrigger("DisgustReaction");
            break;

        case RegionIdentity.Brockmore:
            // Boxer
country[span_16](start_span)[span_16](end_span)[span_17](start_span)[span_17](end_span).
Immediate, brutal combat state.
            SetState(PedestrianState.Hostile);
            Debug.Log("[Audio Bark]: 'Forty-nine and oh, baby! Let's
go!'[span_18](start_span)"[span_18](end_span));
            animator.SetTrigger("BoxingStance");
            break;
    }

    StartCoroutine(ResumeWalkingAfterBump());
  }

  private IEnumerator ResumeWalkingAfterBump()
  {
    yield return new WaitForSeconds(3f);
    if (currentState != PedestrianState.Hostile && currentState !=
PedestrianState.WeatherPanic)
    {
      agent.isStopped = false;
      SetState(PedestrianState.Wander);
    }
  }
}
```

### 3. Audio & Dialect Integration Notes

The code above triggers placeholder Debug.Log statements, but the audio engine behind this is vital. Your audio implementation team must structure the NPC bark database to aggressively

filter by dialect:
 * **The Non-Rhotic Pool:** 95% of the voice lines (assigned to Cascade, New Sefton, and Brockmore) must drop the letter "R" (e.g., "Pahk the cah," "wadah").
 * **The Hard-R Pool:** The 5% of voice lines assigned specifically to the Cape Cobb region (and particularly to characters parodying the "Chip Worthington III" archetype) must intentionally, smugly pronounce every "R" to create immediate class friction.